

Review Form - Requirements Phase Defects

Document Title:	GymTrackerPro Inspection / Review	
Reviewer Name:	Hus Lucian-Antonio, Ilieș Ariana, Ielciu Mara, Horvat Aris, Costin Joldeș, Manciu Patricia	
Review date:	07.04.2026	

Crt. No.	Checked Item	Doc. page/line	Comments/ improvements
1	D_01 - Requirements are incomplete.	GymTrackerPro_Problem_Statement.pdf Page: 3 - 6	Authentication Module The AUTH section contains six requirements (AUTH-01 through AUTH-06) that collectively cover all aspects of the authentication subsystem. Login for both user roles, the hashing algorithm and work factor, session creation and destruction on login and logout, unauthenticated-user redirection, role-based post-login routing, and the account creation policy are all present. No authentication scenario was left without a corresponding requirement. <i>No gap was identified.</i> Member Management Module The MEM section contains seven requirements (MEM-01 through MEM-07) that address the full CRUD lifecycle for gym members together with suspend and unsuspend functionality. Field-level constraints are defined for all input fields, covering character lengths, the email format convention, date format,

		and phone number format. The temporary-password generation policy, pagination configuration, searchable fields, sort order, and duplicate email prevention are all explicitly stated. <i>No gap was identified.</i> Member Self-View Module The MSV section contains four requirements (MSV-01 through MSV-04) addressing member profile view, personal session history with pagination and descending sort, cross-member data isolation, and password self-change with explicit complexity rules. The read-only nature of session data for members is clearly stated. <i>No gap was identified.</i> Exercise Management Module The EXM section contains six requirements (EXM-01 through EXM-06) covering full CRUD for exercises plus archive and unarchive with explicit soft-delete semantics. All permitted enum values for muscle group and equipment type are individually listed. Historical reference preservation when archiving is stated. Member read-only access to the exercise catalogue is included as a SHOULD requirement. <i>No gap was identified.</i> Workout Session Management Module
--	--	--

		The WSM section contains eight requirements (WSM-01 through WSM-08) covering creation of a session with exercises, listing, editing session metadata and exercises, managing session exercises individually, and deletion with cascade. Numeric ranges for all session and exercise fields are defined. The at-least-one-exercise constraint and the single-transaction atomicity requirement are both explicitly stated. <i>No gap was identified.</i> Report Generation Module The RPG section contains seven requirements (RPG-01 through RPG-07) fully specifying the progress report. The filter by date range, the summary metrics including the total-volume formula, the per-exercise breakdown with all required fields, the chronological session table, the server-side computation requirement, and the role-specific access control are all present. No metric or access scenario was omitted. <i>No gap was identified.</i> Non-Functional Requirements Eight non-functional requirements (NFR-01 through NFR-08) address performance with a specific page-load target, usability with a minimum screen width, input validation on both sides, password protection at
--	--	--

			rest and in transit, server-side error logging without client exposure, Prisma-managed schema migrations, naming convention adherence, and Docker Compose containerization with three separate services. All major quality attributes relevant to the application context are covered. <i>No gap was identified.</i> Overall Assessment No incompleteness was identified across any of the seven modules. The requirements document provides thorough coverage of both functional and non-functional aspects for the stated scope and both user roles.
2	D_02 - Requirements are missing.	GymTrackerPro_Problem_Statement.pdf Page: 1 - 6	Use Case Coverage All seven use cases (UN-01 through UN-07) are present in the use case table. Each use case covers a distinct capability: managing members, managing exercises, managing sessions, generating reports, member self-view, member profile view, and exclusive administrator write access. The cross-entity operation and the report generation function required by the assignment are both explicitly captured. <i>No gap was identified.</i> Scope Boundaries and Out-of-Scope Clarity The specification explicitly lists what is in scope and what is out

		of scope. Self-registration, payment processing, exercise media embedding, mobile native apps, real-time notifications, and third-party fitness API integration are all excluded. This boundary definition prevents requirements from being considered missing when they are intentionally excluded. The assumptions section further defines the operational context and pre-conditions. <i>No gap was identified.</i> Negative Path and Edge Case Scenarios The specification does not explicitly state the expected system behavior when a suspended member attempts to log in with valid credentials. AUTH-04 covers unauthenticated-user redirection, and MEM-06 defines the suspension flag, but the interaction between an active session belonging to a subsequently suspended member is not addressed. This is a minor gap that leaves behavior in this edge case open to implementation interpretation. It does not affect the positive-path requirements and is unlikely to cause a functional defect in practice. Overall Assessment No significant requirements are missing given the defined scope.
--	--	--

			The single observation about the suspended-member login edge case is minor and does not constitute a functional requirement gap for the primary use paths.
3	D_03 - Requirements are incorrect.	GymTrackerPro_Problem_Statement.pdf Page: 4 - 5	Typographical Error in EXM-01 The requirement EXM-01 on page 4 uses the word 'execise' instead of 'exercise' in its description. This is a typographical defect that is cosmetic in nature. The meaning of the requirement is unambiguous despite the misspelling, and no implementer would be misled by it. It should nonetheless be corrected in the document for professional quality. Date Format Notation Inconsistency MEM-01 on page 3 specifies date input fields using the YYYY-MM-DD ISO format. WSM-01 on page 5 and RPG-01 on page 6 specify their respective date inputs using DD-MM-YYYY. Both notations refer to dates entered into the system by users. The inconsistency in notation across modules does not create a logical contradiction, but it introduces ambiguity about which format the implementing team should adopt for date picker inputs and backend validation. All other requirements are factually correct and logically consistent.

			Overall Assessment No requirements were found to be logically incorrect or to contradict each other or the stated problem. The observations recorded are a cosmetic spelling error in EXM-01, a notation inconsistency in date formats across modules, and a missing explicit requirement for the behavior of suspended members with active sessions. None of the existing requirements need to be withdrawn or substantially rewritten. However, a new requirement, AUTH-07, shall be added to address the suspended session gap.
4	D_04 - Initialization of the system state has not been considered.	GymTrackerPro_Problem_Statement.pdf Page: 3	Initial State Table The specification includes a dedicated Initial State section on page 3 containing a two-row table. The first row describes the database state: all pending Prisma migrations are applied, and all tables (Exercise, Member, WorkoutSession, and SessionExercise) are initially empty. The second row describes the member state: a seed script creates one administrator account with the specific credentials admin@gymtrackerpro.com and the password admin. This documents both the structural state (schema applied, data absent) and the bootstrapped state (one usable account present) that the system requires before first use. <i>No gap was identified.</i>

			Deployment Assumptions as Pre-conditions The assumptions section states that one deployment instance is expected per gym and that the PostgreSQL database is provisioned and accessible before the application's first launch. These two statements define the infrastructure pre-conditions that must hold for the described initial state to be achievable. Together with the initial state table, they form a complete picture of system initialisation. <i>No gap was identified.</i> Overall Assessment System initialisation is fully and clearly specified. No aspect of the startup state was found to be unconsidered.
5	D_05 - The functions have not been defined adequately.	GymTrackerPro_Problem_Statement.pdf Page: 1 - 6	Use Case Definition Quality All seven use cases include three columns: actor, action need, and business rationale. Each action is stated at a level of specificity that allows it to be directly mapped to one or more functional requirements. The cross-entity use case UN-03 explicitly states that sessions must be recorded atomically and that no data must be lost. The report use case UN-04 includes the date-range filter in its need statement. No use case is vague or circular in its definition. <i>No gap was identified.</i>

		Functional Requirement Precision Each functional requirement states the permitted action, the entity it applies to, and the constraints on inputs or outputs. Pagination size is defined as ten elements per page throughout. Sort orders are specified as ascending or descending with the sort field named. Search scope is defined per module. The at-least-one-exercise constraint for session creation and the atomicity requirement are each given their own dedicated requirement identifier rather than being embedded as a note inside another requirement. <i>No gap was identified.</i> Cross-Entity Function Definition The cross-entity operation that links the Member, Exercise, and WorkoutSession entities is described in the overview as a single atomic action and is formalized across WSM-01 through WSM-04. WSM-03 defines the minimum-exercise constraint, and WSM-04 defines the atomicity requirement. This satisfies the assignment mandate for one multi-entity functionality and provides enough detail for correct implementation. <i>No gap was identified.</i> Overall Assessment All functions are adequately
--	--	---

			defined throughout the document. The level of constraint detail, the use of dedicated requirement identifiers for invariants, and the clear mapping between use cases and requirement modules are all commendable.
6	D_o6 - The user needs are inadequately stated.	GymTrackerPro_Problem_Statement.pdf Page: 1 - 2	Problem Motivation in the Overview The overview section articulates three concrete pain points experienced by small gyms relying on paper logs or spreadsheets: no traceability of member performance over time, no way to identify which exercises are most frequently used or most improved upon, and no structured view of session frequency or training intensity per member. These three pain points directly motivate each of the five capabilities listed immediately below the overview. The connection between the business problem and the solution is explicit. <i>No gap was identified.</i> Administrator User Needs The gym administrator's needs are captured across five use cases with meaningful business rationale. Managing members (UN-01) serves the need for an accurate and historically complete member base. Standardising the exercise catalogue (UN-02) ensures consistency when logging sessions. Recording sessions

		atomically (UN-03) ensures that no data is lost during the logging process. Generating progress reports (UN-04) enables training effectiveness to be assessed and shared with members. Exclusive write access (UN-07) maintains data integrity across the system. <i>No gap was identified.</i> Member User Needs The gym member's needs are captured in two use cases. UN-05 addresses the need to track personal improvement without other members being able to access one's data, making the privacy requirement an integral part of the use case rationale rather than a separate constraint. UN-06 addresses the need to verify the personal data that the gym holds. Password self-management in MSV-04 covers the member's autonomy over their own account security without requiring administrator intervention. <i>No gap was identified.</i> Overall Assessment User needs are clearly and adequately stated for both roles. The 'So that' rationale column in the use case table is populated with meaningful business justifications for every entry. No user needs relevant to the defined scope appears to have
--	--	---

			been overlooked or vaguely described.
7	D_o7 - Comments	GymTrackerPro_Problem_Statement.pdf Page: 1 - 6	Overall Document Quality The requirements document is professionally written and well structured. The use of MoSCoW prioritization, scoped module identifiers with consistent naming, a use case table with rationale, an in-scope and out-of-scope boundary definition, an explicit assumptions section, and a dedicated initial state table demonstrates a level of rigor. Summary of Observations Three observations were recorded during this review. The first is a typographical error in EXM-01 where the word 'exercise' appears instead of 'exercises'. The second is an inconsistency in date format notation between MEM-01, which uses YYYY-MM-DD, and WSM-01 together with RPG-01, which use DD-MM-YYYY. The third is the absence of an explicit requirement covering the behavior of a suspended member who holds an active session, leaving the expected system response in that scenario open to implementation interpretation. The first two observations are minor and do not affect the logical correctness of any existing requirements. No Formal Defects Found No requirements were identified as incomplete, substantively missing, logically incorrect, or

			inadequately defined. A new requirement, AUTH-07, shall be introduced following the inspection to formally specify that suspended members must be denied access to protected routes.
--	--	--	--

Effort to review document (hours): 1h 45m

Review Form - Unit Design Phase Defects

Document Title:	GymTrackerPro Inspection / Review	
Reviewer Name:	Hus Lucian-Antonio, Ilieș Ariana, Ielciu Mara, Horvat Aris, Costin Joldeș, Manciu Patricia	
Review date:	07.04.2026	

Crt. No.	Checked Item	Doc. page/line	Comments/ improvements
1	D_01 - Is the overall organization of the program clear, including good architectural overview?	GymTrackerPro_Problem_Design.pdf Page: 1 - 3	Five-Layer Architecture The design defines a five-layer architecture consisting of the Domain layer, the Repository layer, the Service layer, the Controller layer, and the Presentation layer. The responsibility of each layer is explicitly described. The Domain layer holds type definitions and re-exported Prisma models. The Repository layer encapsulates all Prisma ORM interaction. The Service layer contains business logic and multi-step operations. The Controller layer handles input validation via Zod schemas and delegates to services via Next.js Server Actions. The Presentation layer renders pages using Next.js server and client components. The dependency direction is strictly unidirectional from outer layers toward inner abstractions, ensuring that ORM details do not reach the service or presentation layers. <i>No gap was identified.</i> Entity Relationship Diagram and Data Integrity Invariants

		The ER diagram covers all six tables in the schema (users, members, admins, exercises, workout_sessions, and workout_session_exercises). Cardinalities and foreign key relationships are documented. A list of fifteen data integrity invariants accompanies the diagram and documents cascade delete rules, the RESTRICT constraint preventing exercise deletion when referenced by session exercises, and the soft-delete semantics for Member.isActive and Exercise.isActive. The design choice to separate the User table (which holds authentication data common to both roles) from the Member and Admin profile tables (which hold role-specific data) is noted and justified, avoiding nullable role-specific columns in a single wide table. <i>No gap was identified.</i> Module Responsibility Table Every source file in the project is listed in structured tables across all five layers. Each entry includes a one-line description of the file responsibility and, for repository and service files, the full public method of signatures with parameter names and types, return types, and the error types thrown. This level of specification removes ambiguity from the implementation phase and provides a traceable
--	--	---

			contract between design and code. Shared component files in the Presentation layer are also listed with their rendered content described. <i>No gap was identified.</i> Overall Assessment The program organization is clear, and the architectural overview is thorough. The layered decomposition, the ER diagram with integrity invariants, and the file-level module responsibility tables together provide a complete picture of the system structure.
2	D_02 - Does the architecture account for all of the requirements?	GymTrackerPro_Problem_Design.pdf Page: 3 - 7	Authentication Requirements All six AUTH requirements are accounted for in the design. The auth-controller and auth-service handle login credential verification. Password hashing at twelve bcrypt rounds is assigned to the user repository's member and admin creation methods. Session creation and destruction are assigned to the iron-session library configured in lib/session.ts. Route protection is provided by the admin and member layout components. Role-based post-login routing is driven by the SessionData.role field returned by the controller. Administrator account creation is restricted to the Prisma seed script. <i>No gap was identified.</i> Member, Exercise, and Workout Session CRUD All CRUD operations for the

		three core entities are mapped to design components. UserRepository covers the full lifecycle of members and admins including creation, retrieval by ID and email, listing with search and pagination, update, suspend and unsuspend, and deletion with cascade. ExerciseRepository covers creation, retrieval, listing with search and filter, update, archive and unarchive, and deletion with the RESTRICT check. WorkoutSessionRepository covers creation with atomic exercise inclusion, retrieval, listing with date range and pagination, metadata-only update, full update with exercise replacement in a transaction, and cascade-aware deletion. <i>No gap was identified.</i> Report Generation and Access Control All RPG requirements are addressed by the ReportService and the report controller. The report service computes total volume, average session duration, per-exercise statistics, and the chronological session details entirely in-memory on the server side. The report controller accepts a member ID and validated date range parameters. The admin report page can supply any member ID while the member report page is restricted to supplying the current user's
--	--	--

			own member ID from the session, enforcing RPG-06 and RPG-07 respectively. <i>No gap was identified.</i> Non-Functional Requirements NFR-03 (dual-side validation) is addressed by Zod schemas in the controller layer that validate all inputs before any service call. NFR-04 (password never stored in plain text) is addressed by bcrypt hashing in the repository layer. NFR-05 (server-side logging without stack-trace exposure) is embedded in the Infrastructure error tier description. NFR-06 (Prisma-managed migrations) is the stated database schema management approach. NFR-07 (naming conventions) is defined and applied throughout the module responsibility tables. NFR-08 (Docker Compose with separate services) is referenced in the deployment design. <i>No gap was identified.</i> Overall Assessment The architecture accounts for every stated functional and non-functional requirement. No requirement was found without a corresponding design component.
3	D_03 - Is a coherent error handling strategy provided?	GymTrackerPro_Problem_Design.pdf Page: 8	Three-Tier Error Classification The design defines three error tiers. The Validation tier covers Zod schema failures in the controller layer, which are

		caught immediately and returned as field-level error maps in the ActionResult failure object, allowing the UI to display inline field-specific error messages. The Business Rule tier covers typed AppError subclass instances thrown by repositories or services, which are caught by the controller's first catch branch and returned as form-level message failures containing the error's message string. The Infrastructure tier covers any unexpected error that is not an AppError subclass, which is caught by the controller's catch-all, a generic message is returned to the client, and console.error is called server-side with the actual error for debugging purposes. <i>No gap was identified.</i> Custom Error Hierarchy The AppError base class extends the native JavaScript Error class and normalises the name property to the constructor name, enabling reliable logging and classification. The five subclasses each map to a distinct business or infrastructure scenario. NotFoundError signals a missing resource. ConflictError signals a uniqueness violation or state conflict. AuthorizationError signals invalid credentials. WorkoutSessionRequiresExercisesError signals the domain invariant that a session must have at least one exercise. TransactionError signals a database-level failure during a multi-step operation. The
--	--	--

			instance of AppError check in controllers catches all five subclasses through the class hierarchy. <i>No gap was identified.</i> Client Protection and NFR-05 Compliance The strategy explicitly states that no stack trace or internal implementation detail is ever sent to the client. The infrastructure tier's generic response message decouples the user-visible error from the internal cause. Console error logging on the server side satisfies NFR-05 (server errors logged server-side without exposing stack traces to the end user). An error flow diagram accompanies the description and illustrates the data path from the throw site to user interface. <i>No gap was identified.</i> Overall Assessment The error handling strategy is coherent, complete, and well-documented. All three tiers are cleanly separated with no overlap. The custom error hierarchy is appropriate for the number and variety of business scenarios in the application.
4	D_04 - Have classic design patterns been considered where they might be incorporated	GymTrackerPro_Problem_Design.pdf Page: 8 - 9	Repository Pattern The Repository pattern is applied at lib/repository/ with a dedicated interface file for each entity alongside a Prisma-backed implementation. Services and

	into the architecture?		controllers depend on the interface type rather than the concrete class. This isolates all ORM knowledge to the repository implementations, enabling mock substitution for testing and ORM substitution without touching upstream layers. The pattern is correctly applied across all three domain entities. <i>No gap was identified.</i> Singleton and Dependency Injection with Composition Root The Singleton pattern is implemented on every repository and service via a static getInstance factory method that lazily creates and caches the single instance. A single shared PrismaClient defined in lib/database.ts is the root of all repository instances, preventing connection pool exhaustion in the Next.js serverless environment. The Composition Root in lib/di.ts wires all singletons together in one place. No service or repository constructs its own dependencies. This makes the entire dependency graph visible and modifiable in a single file. <i>No gap was identified.</i> Interface-Based Programming Every repository and service is defined by a TypeScript interface. Concrete classes implement these interfaces. Controllers and services depend
--	-------------------------------	--	---

		on interface types, not concrete implementations. Combined with the Composition Root approach, this means any implementation can be substituted, including test doubles, without modifying any consumer code. This is particularly valuable for the testing scenarios described in the assignment. <i>No gap was identified.</i> Command Pattern and Strategy Pattern Each Server Action in the controller layer is treated as a discrete named Command that is independently callable, independently validatable, and independently testable. The Strategy pattern is applied in the ReportService, where the in-memory Map and reduce aggregation logic is fully encapsulated within the service. The aggregation strategy can be replaced, for example with a call to a dedicated analytics store, without changing any other component. <i>No gap was identified.</i> Overall Assessment Seven design patterns are documented with their application locations and rationale. All seven are appropriately applied. No missed opportunity for beneficial pattern application was
--	--	--

			identified within the scope of this project.
5	D_05 - Is the name and description of each class clearly reflecting the played role?	GymTrackerPro_Problem_Design.pdf Page: 2 - 7	Class and File Naming Convention All classes follow a consistent naming convention of entity name followed by layer suffix. UserRepository, ExerciseService, and WorkoutSessionController each unambiguously indicate both the entity they manage and the architectural layer they belong to. Interface files follow the same convention with an additional Interface suffix. Schema files use the pattern entity-schema.ts. Domain type files use entity.ts. No ambiguous, abbreviated, or misleading names were found across the entire module list. <i>No gap was identified.</i> Composite Domain Type Names Composite types are named to precisely convey their shape. UserWithProfile indicates a User record with both the Member and Admin relations included for cases where either may be present. MemberWithUser indicates a Member record with its parent User record included. MemberWithUserAndTempPassword extends MemberWithUser with a plaintext temporary password field, used only in the member creation flow where the password must be displayed once to the administrator. Naming makes the purpose and

			shape of each type of self-documenting. <i>No gap was identified.</i> Supporting Type and Utility Names The ActionResult type name accurately describes its role as the uniform return envelope for all Server Actions. SessionData clearly describes the payload of the iron-session cookie. PageResult is a generic wrapper whose name communicates that it carries a page of items plus a total count. The AppError class and its five subclasses (NotFoundError, ConflictError, AuthorizationError, WorkoutSessionRequiresExercisesError, TransactionError) each have names that are self-explanatory without requiring documentation to understand their purpose. <i>No gap was identified.</i> Overall Assessment All class names and descriptions clearly reflect the played roles. The naming is consistent, self-documenting, and free of ambiguity across all layers.
6	D_06 - Is the architecture designed to accommodate likely changes?	GymTrackerPro_Problem_Design.pdf Page: 9	PDF Export for the Progress Report The design notes that adding PDF export capability for the progress report would require only a new pdf-report-adapter.ts file that accepts the existing Report domain object and

		renders it to the desired format. No changes to the ReportService, the report controller, or any presentation component are needed because the Report type already contains all the data fields required for any reasonable rendering of the report content. Database Provider Substitution The design explicitly states that swapping the database provider requires only changing the datasource provider field in schema.prisma and running the Prisma migration command. All other layers are insulated by the repository interfaces and the Composition Root wiring. This claim is verified by the fact that no controller, service, or page component imports Prisma types directly. <i>No gap was identified.</i> REST API Addition Adding REST API endpoints is handled by creating new Next.js route handlers in the app/api/ directory that call the existing controller functions or services. No service or repository changes are required because the controller layer is already decoupled from the transport mechanism used to invoke it. The Server Actions and API route handlers can share the same underlying logic. <i>No gap was identified.</i>
--	--	---

			Multi-Tenancy and Role Expansion The design describes that supporting multiple gyms would require adding a gymId foreign key to the Member, Exercise, and WorkoutSession models and enforcing it in repository findAll and create calls. Service and presentation layers are identified as unaffected. Role expansion requires adding a new value to the Role enum and a corresponding profile model in the Prisma schema, with UserRepository handling the profile creation atomically. The SessionData type already carries a role field and a profile ID, requiring no session mechanism changes. <i>No gap was identified.</i> Overall Assessment The architecture demonstrates systematic consideration of likely future changes. The six scenarios documented in the Accommodating Future Changes section each have a concrete and specific action plan. The pattern choices (Repository interfaces, Composition Root, Dependency Injection) are the primary enablers of this adaptability.
7	D_07 - Comments	GymTrackerPro_Problem_Design.pdf Page: 1 - 9	Overall Design Quality The design document is thorough, well-structured, and consistent with the requirements specification. The documentation of data integrity

		invariants, the three-tier error handling strategy with a dedicated error hierarchy diagram, the per-method repository signatures, and the Accommodating Future Changes section all go beyond what is typically expected at this level and demonstrate genuine software engineering competence. Consistency with Requirements Every functional and non-functional requirement from the requirements specification maps to a clearly identified design component. No requirement was found without a corresponding design decision. The design and requirements of documents are mutually consistent and together form a coherent specification of the system. No Defects Found No structural defects, naming deficiencies, requirement coverage gaps, or architectural coherence issues were identified during this review. The design document is ready to serve as the implementation blueprint without ambiguity.
--	--	---

Effort to review document (hours): 2h 0m

Review Form - Coding Phase Defects

Document Title:	GymTrackerPro Inspection / Review	
Reviewer Name:	Hus Lucian-Antonio, Ilieș Ariana, Ielciu Mara, Horvat Aris, Costin Joldeș, Manciu Patricia	
Review date:	07.04.2026	

Crt. No.	Checked Item	Doc. page/line	Comments/ improvements
1	D_01 - Decision logic or sequencing is erroneous or inadequate.	auth-controller.ts, lines 18-41 auth-service.ts, lines 32-51 workout-session-repository.ts, lines 32-72 and 130-187 report-service.ts, lines 43-109	Authentication Flow Sequencing The login controller validates the incoming data using loginUserSchema.safeParse before making any service call, returning early on validation failure without touching the database. The auth service then looks up the user by email and, if the user exists, compares the provided password against the stored hash using bcrypt.compare. An AuthorizationError with the message 'Invalid email or password' is thrown in both the 'email not found' and 'password mismatch' cases, preventing username enumeration. On successful verification the controller populates the iron-session fields, saves the session, and returns the SessionData. The logout action calls session.destroy and returns a success result. The sequencing is correct throughout. <i>No gap was identified.</i> Workout Session Creation Atomicity The create method in the workout session repository correctly sequences its operations. The empty-exercises guard fires first, before any database access, throwing a WorkoutSessionRequiresExercisesError

			if the exercises array is empty. The member existence check follows, throwing a <code>NotFoundError</code> if the member ID is not found in the database. The actual write is a single Prisma nested create that writes the workout session and all session exercises as one atomic operation inside Prisma's implicit transaction. If the Prisma call throws, the error is caught and wrapped in a <code>TransactionError</code>. This sequence correctly satisfies both WSM-03 and WSM-04 with no logic gaps. <i>No gap was identified.</i> Report Aggregation Logic The <code>getMemberProgressReport</code> method in the report service uses a Map keyed by exercise ID to accumulate per-exercise statistics (sets, reps, and volume) in a single pass over all sessions and their exercises. A parallel Map from exercise ID to a Set of session IDs is maintained to count the number of distinct sessions in which each exercise appears, correctly avoiding double-counting when an exercise occurs more than once in a single session. After the pass, each exercise's <code>sessionCount</code> is assigned from the size of its corresponding Set. Average session duration is computed with a guard that returns zero when the session count is zero, preventing division by zero. All fields required by RPG-02, RPG-03, and RPG-04 are correctly produced. <i>No gap was identified.</i> Update-With-Exercises Transaction Sequencing The <code>updateWithExercises</code> method opens
--	--	--	--

			an interactive Prisma transaction and correctly handles three mutually exclusive cases for session exercises. Exercises not present in the new input (identified by computing the difference between the set of existing row IDs and the set of IDs present in the input) are deleted with deleteMany. Exercises with an ID present in the input are updated with individual update calls. Exercises without an ID (newly added rows) are inserted with createMany. The WorkoutSessionRequiresExercisesError guard fires before the transaction is opened, ensuring the commit will never produce a session with zero exercises. The session metadata update is performed last inside the transaction, ensuring exercise consistency precedes the session record update. No sequencing errors were found. <i>No gap was identified.</i> Overall Assessment No decision logic or sequencing errors were identified anywhere in the codebase. All critical flows follow correct ordering of guards, existence checks, and write operations.
2	D_02 - Branching is erroneous.	app/admin/layout.tsx , lines 12-14 app/member/layout.tsx, lines 7-10 user-repository.ts, lines 35-40 and 216-223 workout-sessions/[id]/page.tsx (member), lines 35-37	AppError Handling Consistency Across All Controllers All five controller files (auth-controller.ts, user-controller.ts, exercise-controller.ts, workout-session-controller.ts, and report-controller.ts) follow an identical branching pattern. After input validation, each service call is wrapped in a try-catch block whose first branch checks whether the caught error is an instance of AppError. If it is, the controller returns an ActionResult failure with the error's message string. If it is not, the catch-all

			branch returns a generic failure message. Because all five concrete error subclasses (NotFoundError, ConflictError, AuthorizationError, WorkoutSessionRequiresExercisesError, and TransactionError) extend AppError through the class hierarchy, the instance of AppError check correctly routes all of them. No controller deviates from this pattern. <i>No gap was identified.</i> Member Layout Missing Authentication Guard (minor observation) The admin layout in app/admin/layout.tsx checks that session.userId is present and that session.role equals Role.ADMIN before rendering the admin interface, and it calls redirect('/login') if either condition fails. The member layout in app/member/layout.tsx does not perform an equivalent check. Individual member pages compensate by checking session.memberId before rendering and returning null if it is absent, which prevents data exposure. However, returning null results in a blank rendered page rather than a redirect to the login page, which is a partial divergence from the AUTH-04 requirement that states unauthenticated users should be redirected to the login page for any protected route. No member data is exposed by this omission. Admin-side protection is not affected. Member Data Isolation Enforcement The workout session detail page for members checks that the fetched workout session's memberId field matches the session's own memberId before rendering. If they differ, the page returns null immediately, preventing
--	--	--	---

		cross-member data access and satisfying MSV-03. The member report page passes session.memberId directly as the memberId argument to getMemberProgressReport, making it structurally impossible for a member to request another member's report through the member interface. The member workout sessions list page scopes its listMemberWorkoutSessions call to the current user's memberId from the session. Data isolation is consistently and correctly enforced. <i>No gap was identified.</i> Email Uniqueness Conflict Branch In user-repository.ts, both createMember and updateMember include an explicit email uniqueness check before performing any write. For createMember, a findUnique call is made on the email field and a ConflictError is thrown if a result is returned. For updateMember, the same check is performed but is correctly skipped when the new email value is equal to the current email value, preventing a false conflict when the administrator submits an update form without changing the email field. ConflictError propagates through the service layer and is caught by the controller's AppError branch, returning an appropriate error message. The branching is correct. <i>No gap was identified.</i> Overall Assessment Branching logic is correct throughout the codebase. The single minor observation regarding the member layout's missing redirect guard is noted but does not
--	--	--

			expose data or compromise system security.
3	D_03 - There are undefined loop terminations.	user-service.ts, lines 56-63 report-service.ts, lines 46-83 workout-session-repository.ts, lines 88-109 and 141-165	Shuffle in Temporary Password Generation The shuffle loop in generateTempPassword iterates the variable i from combined.length minus one (which equals fifteen for a sixteen-character password) down to one, decrementing by exactly one on each iteration. The loop body executes exactly fifteen times and always terminates. The three required characters (one uppercase, one digit, and one special character) are placed at fixed positions in the combined array before the shuffle begins, ensuring that the password complexity policy required by MEM-02 is satisfied regardless of how the shuffle rearranges the remaining characters. Crypto.getRandomValues is called with a fresh single-byte Uint8Array on each iteration, providing cryptographically secure random values. <i>No gap was identified.</i> Report Aggregation Iteration In report-service.ts the outer sessions.map call iterates over the array of workout sessions returned by the repository's findAll call, which is a finite Prisma result array. The inner session.exercises.map call iterates over the exercises array of each session, also a finite Prisma result. Both calls use Array.prototype.map, which terminates by definition for any finite array. The exerciseBreakdown sort operates on the array of values extracted from the exercise statistics Map, which is bounded by the number of distinct exercises in the result. The final reduce calls for total volume and total duration operate on

			the finite sessionDetails array. No unbounded iteration exists. <i>No gap was identified.</i> Paginated Query and Transaction Exercise Loops The findAll method in the workout session repository uses Prisma's built-in take and skip parameters for pagination without any manual iteration. The updateWithExercises method contains a for-of loop over the exercises input array filtered to those with an ID field. This array is bounded by the exercises input array, which the Zod schema validates to contain at least one element. The deleteMany and createMany operations receive arrays derived from the same input, also bounded. No loop in the codebase has an undefined termination condition. <i>No gap was identified.</i> Overall Assessment No undefined loop terminations were found anywhere in the codebase. All iterative constructs operate on finite, bounded collections with deterministic termination.
4	D_04 - I/O format error exists.	lib/schema/utils.ts lib/schema/workout-session-schema.ts, lines 6-8 lib/schema/user-schema.ts, lines 19-22 lib/schema/report-schema.ts user-repository.ts (date conversion) workout-session-repository.ts (date conversion)	Date Format Notation Inconsistency Between Requirements and Implementation The isoDateRegex defined in lib/schema/utils.ts validates dates using the YYYY-MM-DD ISO 8601 format. This regex is consistently imported and applied in user-schema.ts for the dateOfBirth and membershipStart fields, in workout-session-schema.ts for the session date field, and in report-schema.ts for the startDate and endDate filter fields. The requirements

		specification states DD-MM-YYYY as the expected date input format in WSM-01 and RPG-01, while MEM-01 uses YYYY-MM-DD. In practice the HTML date input element natively produces and consumes ISO format, making the YYYY-MM-DD implementation the functionally correct choice for a web application. The discrepancy is therefore between the requirements of text notation and the implementation and does not cause any runtime format error or data loss. It is a notation inconsistency worth documenting. Numeric Field Coercion Duration, sets, reps, and weight in workout-session-schema.ts all use z.coerce.number, which correctly converts the string values produced by HTML form submissions to numbers before applying the min and max range validators. Weight is declared as Float in the Prisma schema and is correctly handled as a floating-point JavaScript number throughout the application. In report-service.ts the weight value from a Prisma session exercise row is explicitly converted with Number(entry.weight) before use in volume calculations, accounting for the Decimal type that Prisma may return from a Float database column. <i>No gap was identified.</i> Date String to JavaScript Date Conversion In user-repository.ts the dateOfBirth and membershipStart string values from validated inputs are converted to JavaScript Date objects using new Date(value) before being passed to Prisma, which requires DateTime
--	--	---

			columns to receive Date objects. The same conversion is applied in workout-session-repository.ts for the session date field in both the create and update methods. The conversion is consistently applied at the correct layer (repository) and produces no format errors given that the input strings have been validated as YYYY-MM-DD format by Zod before reaching the repository. <i>No gap was identified.</i> Overall Assessment No I/O format errors were found in the implementation. The date format notation inconsistency between the requirements text and the schema validation is documented as a minor observation. All numeric and date conversions are correctly handled at the appropriate layer.
5	D_05 - Subprogram invocations are violated.	lib/di.ts lib/controller/auth-controller.ts lib/controller/user-controller.ts lib/controller/workout-session-controller.ts lib/repository/workout-session-repository.ts	Dependency Injection Wiring in lib/di.ts The composition root correctly instantiates all repositories with the single shared Prisma client from lib/database.ts. UserRepository, ExerciseRepository, and WorkoutSessionRepository are created once. AuthService and UserService each receive userRepository as their sole dependency. ExerciseService receives exerciseRepository. WorkoutSessionService receives workoutSessionRepository. ReportService receives both workoutSessionRepository and userRepository, correctly matching its constructor signature which requires both in order to fetch sessions and resolve member details for the report. No dependency is missing, duplicated, or provided to the wrong constructor. No circular dependencies exist.

			<i>No gap was identified.</i> Controller-to-Service Delegation Purity All controller functions validate their inputs with Zod and then delegate to the corresponding service method without introducing any business logic. No raw Prisma access or database query occurs in any controller file. The <code>createWorkoutSession</code> controller, for example, validates session data and exercise data separately, then calls <code>workoutSessionService.createWorkoutSession</code> with the validated typed inputs. The <code>updateWorkoutSessionWithExercises</code> controller validates both the session metadata and the exercise list before delegating to the service. The layering contract is consistently respected. <i>No gap was identified.</i> Service-to-Repository Delegation Purity Service methods delegate to repository methods without adding logic that belongs at the repository level. Data integrity invariants such as email uniqueness checks, resource existence checks, and transaction management are all enforced within the repository implementations where direct database access is permitted. No service method constructs Prisma query objects or calls the database client directly. The service layer's role as a thin orchestration and business-logic layer is consistently maintained. <i>No gap was identified.</i> Prisma Query Structure Correctness Nested creates in <code>workout-session-</code>
--	--	--	--

			repository.ts use the correct Prisma relation name (exercises with a nested create array mapping to WorkoutSessionExercise fields). Include directives for nested relations follow the two-level pattern required to include session exercises and their catalogue exercise entries simultaneously. The interactive transaction in updateWithExercises uses the tx parameter correctly, ensuring all operations within the callback execute against the same transaction client. The batch transaction in findAll correctly groups findMany and count in a two-element array passed to database.\$transaction. <i>No gap was identified.</i> Overall Assessment No subprogram invocation violations were found. All method calls are correctly typed, all dependencies are correctly wired, and the layering contract is respected throughout the codebase.
6	D_o6 - There is confusion in the use of parameters.	lib/domain/action-result.ts lib/service/auth-service.ts, lines 48-50 lib/schema/workout-session-schema.ts, lines 73-76 app/admin/layout.tsx and app/member pages lib/session.ts	ActionResult Discriminated Union Usage The ActionResult type is a discriminated union where the success boolean field is the discriminant. When success is true, the data field of generic type T is present and populated. When success is false, the message field is present as a string, and the optional errors field may carry a map of field names to error message arrays for Zod validation failures. All page components that call controller functions correctly check the success field before accessing the data field. No instance of direct data field access without a prior success check was found in any of the examined pages. The distinction between field-level errors (for validation failures) and form-level

			messages (for business rule failures) is consistently applied across all controller functions. <i>No gap was identified.</i> Optional ID Field in WorkoutSessionExerciseUpdateInput The optional id field in the WorkoutSessionExerciseUpdateInput type is used in the update-with-exercises repository method to distinguish between two semantically different cases. An exercise entry that carries an id value is an existing row to be updated. An exercise entry without an id value is a new row to be inserted. The filter calls in the repository method (exercises filtered to those with an id for updates, and exercises filtered to those without an id for creates) are correctly structured and do not overlap. The distinction is applied consistently, and the intent of the optional field is clear from both the schema comment and the repository usage. <i>No gap was identified.</i> Optional SessionData Fields and Null Safety SessionData defines memberId and adminId as optional string fields because a given authenticated user occupies exactly one role. In auth-service.ts these fields are populated using optional chaining (user.member?.id and user.admin?.id) so that a missing relation returns undefined rather than throwing. In the admin layout, session.fullName uses optional chaining with a nullish coalescing fallback to the string 'A' to
--	--	--	---

			safely handle any edge case where <code>fullName</code> is absent. The only non-null assertion operator in the codebase is used for <code>process.env.IRON_SESSION_SECRET</code> in <code>lib/session.ts</code>. This is appropriate because the Docker Compose configuration enforces the presence of this variable at container startup using a required substitution expression, making the assertion safe by deployment contract. <i>No gap was identified.</i> Overall Assessment No confusion in the use of parameters was found. TypeScript interfaces and Zod-inferred input types provide strong typing at all boundaries, and all optional fields are handled with appropriate null-safety patterns.
7	D_07 - Comments	Full source code under <code>gymtrackerpro/</code> directory	Overall Code Quality The codebase is of high quality and follows the design specification closely. TypeScript strict typing through repository and service interfaces and Zod-inferred input types provides strong type safety at all system boundaries. The separation of concerns between layers is consistently maintained throughout all files. The codebase is clean, well-organised, and readable. Security Practices Passwords are hashed using <code>bcrypt</code> with twelve rounds before storage, satisfying AUTH-02. The session cookie is configured as <code>httpOnly</code> and <code>secure</code> in production, satisfying NFR-04. Temporary passwords are generated using <code>crypto.getRandomValues</code>, which provides cryptographically secure randomness as required by MEM-02. The

		uniform error message for invalid login credentials prevents username enumeration. No sensitive data including password hashes or stack traces is included in any ActionResult response returned to the client, satisfying NFR-05. Summary of Observations Two minor observations were identified during the coding phase inspection. The first is that the member layout does not include an explicit authentication guard equivalent to the one present in the admin layout, resulting in a blank page rather than a redirect to the login page for unauthenticated visitors to member routes. The second is the date format notation inconsistency between the requirements text (DD-MM-YYYY in WSM-01 and RPG-01) and the implementation schemas (YYYY-MM-DD throughout). Neither observation constitutes a formal defect. No Formal Defects Found No decision logic errors, erroneous branches that expose data, undefined loop terminations, I/O format errors, subprogram invocation violations, or parameter confusions were identified in the codebase. The implementation is consistent with the design specification.
--	--	---

Effort to review document (hours): 2h 20m